

From $\text{\LaTeX}$ to Quarto

Scientific Writing, Reproducible Publishing, and Beyond

Imad El Badisy, PhD

2026-06-24

1 1. Part 1 - Scientific Writing with $\text{\LaTeX}$

2 2. Part 2 - Quarto for Reproducible Publishing

Section 1

1. Part 1 - Scientific Writing with $\text{\LaTeX}$

1.1 A Brief History

T_EX - D. Knuth, 1978

- Underlying engine, still used today
- Deterministic, reproducible output

L^AT_EX - L. Lamport, 1984

- Macro layer on top of T_EX
- Separates content from formatting
- Became the standard for scientific writing

Year	Event
1978	T _E X released
1984	L ^A T _E X 2.09
1994	L ^A T _E X2e (current)
1999	pdfT _E X → direct PDF
2004	XeT _E X → Unicode
2007	LuaT _E X → scripting
2012	Overleaf launched

1.2 L^AT_EX is not Overleaf!

For many students, **Overleaf** = **LaTeX** but they are not the same thing!



L^AT_EX (1984)

- A typesetting *language + compiler*
- Runs anywhere: machine, server, cloud
- Free and open source
- Works offline in any editor

Overleaf (2012)

- A *web app* built on top of L^AT_EX
- Runs the compiler on their servers
- Adds: browser UI, collaboration, cloud storage
- You are still writing L^AT_EX

Analogy: L^AT_EX is the engine. Overleaf is the car. Swap the car - the engine stays the same.

1.3 Why $\LaTeX$?

The problem with Word

- Copy-paste figures from R $\rightarrow$ manual update on every change
- `final_v3_FINAL_revised.docx`
- Math typesetting is painful

Feature	Word	$\LaTeX$
Typesetting	Medium	Excellent
Math	Poor	Native
Figure placement	Manual	Algorithmic
Bibliography	Plugin	Built-in
Version control	Hard	Easy
Journal templates	Limited	Extensive

1.4 Where to Write $\text{\LaTeX}$ - Platforms

Platform	Type	Best for
Prism (OpenAI)	Browser	Beginners + AI writing
Overleaf	Browser	Collaboration
TeXStudio	Desktop	Full-featured editor
VS Code + LaTeX Workshop	Desktop	Programmers / Git
TeXmaker	Desktop	Lightweight editor
RStudio	Desktop	R users

Recommendation: beginners $\rightarrow$ Prism or Overleaf; advanced $\rightarrow$ VS Code + Git.

1.5 Why Document Classes?

`\documentclass` is the first decision - it sets the layout, section hierarchy, and all defaults.

Class	Use	LaTeX Command
		% Generic paper <code>\documentclass[12pt]{article}</code>
article	Papers, short reports	% Two-column journal <code>\documentclass[twocolumn]{article}</code>
report	Theses, longer reports	% Elsevier submission <code>\documentclass{elsarticle}</code>
book	Books, front/back matter	% Slide deck <code>\documentclass{beamer}</code>
beamer	Slides	<code>\usetheme{Madrid}</code>
elsarticle	Elsevier journals	
IEEEtran	IEEE journals	

Why it matters: `\section{}` renders differently across classes. Always start from the journal's .cls file.

The class file controls everything - margins, fonts, title format, header/footer.

1.6 Document Structure

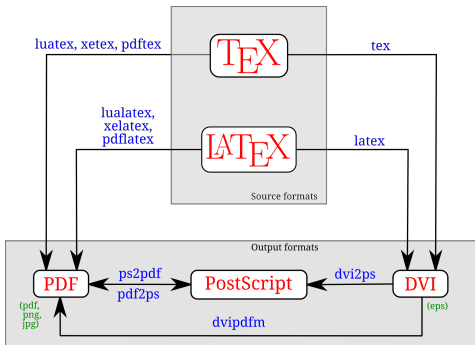
```
\documentclass[12pt, a4paper]{article}    % (1) class

% (2) Preamble - packages and settings
\usepackage{amsmath, amssymb}           % math
\usepackage{graphicx}                   % figures
\usepackage{booktabs}                   % tables
\usepackage{hyperref}                   % links
\usepackage[style=apa, backend=biber]{biblatex}
\addbibresource{references.bib}

\title{My Paper}
\author{Author Name}
\date{\today}

% (3) Body
\begin{document}
\maketitle
```

1.7 How to Compile



```
# pdflatex (most common)
pdflatex paper.tex
biber paper           # bibliography
pdflatex paper.tex    # 2nd pass:
```

```
# xelatex (Unicode + fonts)
xelatex paper.tex
```

```
# latexmk (recommended)
# automates all passes
latexmk -xelatex paper.tex
```

Run at least twice -
cross-references are resolved on the
second pass.

Overleaf handles this
automatically.

1.8 Mathematical Notation

Inline: $\hat{\beta} = 0.42 \rightarrow$

$\hat{\beta} = 0.42$

Display:

```
\begin{equation}
  \hat{\beta} = (X^{\top} X)^{-1} X^{\top} y
\end{equation}
```

Aligned:

```
\begin{align}
  \ell(\theta) &= \sum_{i=1}^n \log f(y_i | \theta) \\
  \hat{\theta} &= \arg\max_{\theta} \ell(\theta)
\end{align}
```

Custom commands (define once):

```
\newcommand{\E}{\mathbb{E}}
\newcommand{\Var}{\mathrm{Var}}
\newcommand{\iid}{\overset{iid}{\sim}}
```

Symbol	L ^A T _E X
$\sum$	<code>\sum</code>
$\prod$	<code>\prod</code>
$\int$	<code>\int</code>
$\hat{\theta}$	<code>\hat{\theta}</code>
α, β	<code>\alpha, \beta</code>
$\mathbb{E}[X]$	<code>\mathbb{E}[X]</code>
$\leq, \geq$	<code>\leq, \geq</code>
$\in, \notin$	<code>\in, \notin</code>
$\sim$	<code>\sim</code>
$X^{\top}$	<code>X^{\top}</code>

Demo $\rightarrow$

[demos/latex/02-math/](#)

1.9 Diagrams - TikZ and Alternatives

TikZ - standard $\text{\LaTeX}$ diagram package

```
\usepackage{tikz}
\usetikzlibrary{arrows.meta, positioning}

\begin{tikzpicture}[node distance=1cm,
  box/.style={draw, rounded corners,
    fill=blue!8, minimum width=2.8cm}]
  \node[box] (A) {Inclusion};
  \node[box, below=of A] (B) {Eligible};
  \node[box, below=of B] (C) {Analysis};
  \draw[->] (A)--(B); \draw[->] (B)--(C);
\end{tikzpicture}
```

Other uses: DAGs, survival trees, neural nets, timelines.

1.10 Debugging with AI Coding Agents

Modern AI agents (Claude, Codex, GitHub Copilot, Cursor) accelerate $\text{\LaTeX}$ and Quarto debugging.

What to give the agent:

- 1 The **exact error message** from the terminal or log
- 2 The **minimal code block** that triggers the error
- 3 The **context**: document class, relevant packages, what you expected

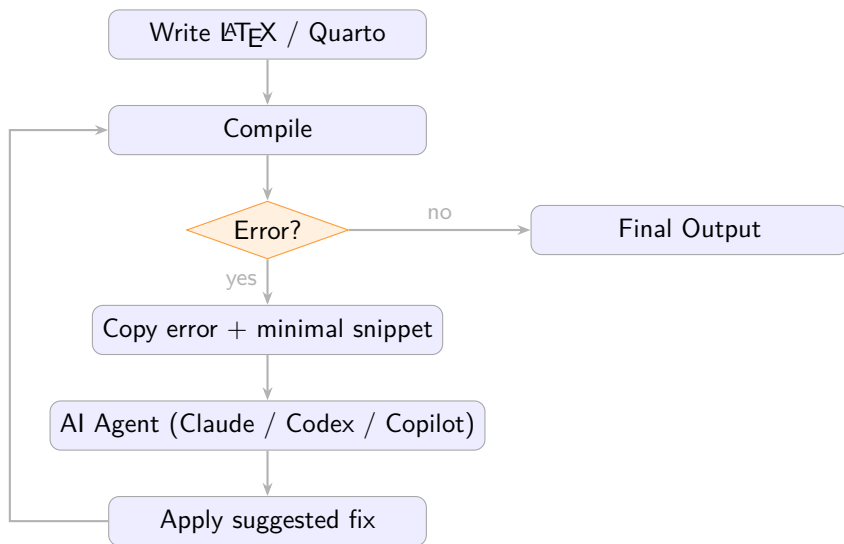
Effective prompt:

I am writing LaTeX with `\documentclass{article}` and `\usepackage{}`
The following snippet fails to compile:

```
\begin{align}
  y = x^2
\end{align*}
```

Error: "Extra `\begin{align}`."

1.11 AI-Assisted Debugging Workflow



Demo → `demos/latex/` - 01-minimal, 02-math, 03-template, 04-bibliography

Section 2

2. Part 2 - Quarto for Reproducible Publishing

2.1 From $\text{\LaTeX}$ to Quarto - Why?

$\text{\LaTeX}$	Quarto
Excellent typesetting	Same $\text{\LaTeX}$ engine underneath
Manual copy-paste of results	Results computed inside the document
One output format	HTML, PDF, slides, website, book
Text only	Code + text unified
Steep learning curve	Markdown + YAML + code

Quarto = $\text{\LaTeX}$ quality + reproducible code + multiple outputs from one source

2.2 What Quarto Solves

Quarto turns one plain-text `.qmd` source into many publishable outputs.

Need	Output format
Web report / dashboard	HTML
Submission / print	PDF (via $\text{\LaTeX}$)
Lecture slides	RevealJS (browser) or Beamer (PDF)
Course or research site	Website
Thesis / textbook	Book $\rightarrow$ HTML + PDF + EPUB
Repeated journal format	Template / extension

One source file $\rightarrow$ many targets. No copy-paste.

2.3 Compilers and Install

Install check:

```
quarto --version  
quarto check      # diagnose engines
```

Render commands:

```
quarto render report.qmd  
quarto render report.qmd --to pdf  
quarto render report.qmd --to html  
quarto preview report.qmd # live
```

`quarto check` shows which engines are installed and working.

2.4 File Anatomy

Every .qmd file has three layers:

```
---  
title: "My Report"           # YAM  
author: "Student Name"  
format:  
  html:  
    toc: true  
    code-fold: true  
  pdf:  
    number-sections: true  
execute:  
  warning: false  
bibliography: refs.bib  
---
```

```
## Introduction      ← Markdown block
```

```
This is bold and italic.
```

- First item
- Second item

```
```${r}                ← Code chunk  
#| label: fig-km
#| fig-cap: "KM curve."
survfit(...) |> plot()
```
```

Rule: write once → render to any format.

2.5 Markdown - The Prose Layer

Markdown (J. Gruber, 2004) - lightweight markup, readable as plain text, converts to HTML/PDF via Pandoc.

- In Quarto: Markdown is the prose layer. $\text{\LaTeX}$ runs underneath for PDF.

Syntax comparison

| Element | $\text{\LaTeX}$ | Markdown |
|---------|-------------------------------|------------------------|
| Bold | <code>\textbf{x}</code> | <code>**x**</code> |
| Italic | <code>\textit{x}</code> | <code>*x*</code> |
| Section | <code>\section{T}</code> | <code>## T</code> |
| Bullet | <code>\begin{itemize}</code> | <code>- item</code> |
| Link | <code>\href{u}{t}</code> | <code>[t](u)</code> |
| Code | <code>\texttt{x}</code> | <code>`x`</code> |
| Image | <code>\includegraphics</code> | <code></code> |

2.6 YAML - The Control Layer

YAML controls the entire output without touching the content.

```
---
title: "Survival Analysis Report"
author: "Imad El Badisy"
date: today
format:
  html:
    toc: true
    theme: cosmo
  pdf:
    toc: true
    number-sections: true
    pdf-engine: xelatex
execute:
  message: false
bibliography: references.bib
---
```

2.7 Code Chunks and Inline Code

Chunk options (`#|` syntax - same for R, Python, Julia):

| Option | Effect |
|----------------------|--|
| <code>label</code> | names output for <code>@fig-</code> refs |
| <code>echo</code> | show / hide source code |
| <code>eval</code> | run / skip chunk |
| <code>fig-cap</code> | figure caption |
| <code>tbl-cap</code> | table caption |
| <code>warning</code> | show / hide warnings |

2.8 Cross-References and Citations

Figures, tables, equations - use label prefixes:

```
! [Flow diagram] (flow.png) {#fig-flow}      → cite as @fig-flow  
: Patient groups {#tbl-groups}                → cite as @tbl-groups  
$$ h(t|x) = h_0(t)e^{\{x^{\top}\beta\}} $$ {#eq-cox} → @eq-cox
```

Citations - add .bib in YAML, then cite inline:

```
bibliography: references.bib
```

```
The Cox model [Cox1972] is widely used in survival analysis.
```

Label prefixes are mandatory: fig-, tbl-, eq-, sec- - wrong prefix = broken reference.

Demo → `demos/quarto/02-crossrefs/`

2.9 Output Formats

| Format | YAML key | Render command | Best for |
|-----------------|-----------------|----------------|------------------------|
| HTML | html | --to html | Web, interactive |
| PDF | pdf | --to pdf | Submission, archiving |
| Word | docx | --to docx | Collaborators |
| Beamer slides | beamer | --to beamer | PDF lectures |
| RevealJS slides | revealjs | --to revealjs | Browser talks |
| Website | project website | quarto preview | Course / research site |
| Book | project book | quarto render | Thesis, textbook |
| EPUB | epub | --to epub | Mobile reading |

All formats share one source - switching is a YAML change, not a rewrite.

2.10 Slides - RevealJS vs Beamer

RevealJS - browser slides

```
format:
  revealjs:
    theme: default
    slide-number: true
    incremental: true
```

- Runs in any browser
- Animations, fragments, speaker notes
- Easy to share: just a URL
- Interactive elements (Observable)

Structure: with `slide-level: 2`, # headings → sections, ## headings → slides.

Demo → `demos/quarto/03-slides/` and `demos/quarto-lecture/02-slides/`

Beamer - PDF slides

```
format:
  beamer:
    theme: Madrid
    colortheme: crane
    pdf-engine: xelatex
```

- Classic conference / print format
- Full $\text{\LaTeX}$ math and TikZ
- Works offline, always renders the same

2.11 Projects and _quarto.yml

Use a **project** when you have more than one file.

```
my-project/  
_quarto.yml  
index.qmd  
methods.qmd  
results.qmd  
references.bib  
figures/
```

```
quarto render      # all files  
quarto preview     # live reload  
quarto create project book n  
quarto create project website
```

```
# _quarto.yml - shared config  
project:  
  type: website      # or: book, document  
  output-dir: _output  
website:  
  title: "My Course"  
  navbar:  
    left:  
      - index.qmd  
      - methods.qmd  
format:  
  html:  
    toc: true  
    theme: cosmo  
  pdf:  
    number-sections: true
```

2.12 Templates

Use templates when the same structure repeats across documents.

Apply a template:

```
quarto use template owner/repository
```

Workflow:

- 1 Render unchanged - verify it compiles
- 2 Edit YAML (title, author, date)
- 3 Render again
- 4 Replace body content
- 5 Final render

Useful repos:

- quarto-journals - journal formats
- quarto-thesis - thesis template
- quarto-revealjs-clean -

Create your own template:

```
my-template/  
  template.qmd      ← example document  
  _extensions/  
    my-template/  
      _extension.yml  
      custom.tex     ← LaTeX preprocessor  
      theme.scss     ← HTML theme
```

A template is just a .qmd file with a custom format - share it with `quarto use template`.

Demo →

demos/quarto-lecture/05-templates

2.13 Lab - Build Your Own Templates

3 hours remaining - build from scratch in pairs or solo:

| Task | Output | Key concepts |
|----------------------------------|--------------------|--|
| 1 - Minimal report | HTML + PDF | YAML, Markdown, code chunk |
| 2 - Slide deck | RevealJS or Beamer | slide-level, themes |
| 3 - Website | HTML site | _quarto.yml, navbar |
| 4 - Book / thesis chapter | HTML + PDF | chapters, cross-refs |
| 5 - eBook | EPUB | epub format, covers |
| 6 - Custom template | reusable format | _extensions/, custom L ^A T _E X |

Start from demos/quarto-lecture/ - one folder per task.